

Cell-Level Inheritance Verification: Applying the Standard Operational Test Suite at Cell Scope in CKS-Governed AI Selves

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 8, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new architectural commitments. Its sole contribution is to formalize one operational variant of Paper 2's cell concept: **cell-level inheritance verification**, the operational mechanism by which the recursive Paper 1 commitments hold at cell scope in Paper 2 deployments, applying Series A's standard operational test suite (A5.01–A5.16) at cell level. This note closes the four-note decomposition of B1.03 (cell as atomic unit), following B2.11 (cell as modular unit — external interfaces), B2.12 (cell-internal architecture), and B2.13 (cell-to-cell relationships within aspects).

Abstract

Paper 2 commits that all of Paper 1's architectural commitments hold at every level of the three-level cell-aspect-Self structure, and a separate note (B1.20) formalizes that recursive inheritance as an architectural commitment. The architectural claim leaves a distinct operational question: how a deployment establishes that Paper 1's commitments actually hold for any specific cell, rather than only that the architecture promises they should. This note formalizes the operational answer at cell scope: cell-level inheritance verification is the application of Series A's standard sixteen-test operational suite (A5.01–A5.16) at cell scope, with each test verifying one Paper 1 commitment for the cell as architectural artifact. The note states the verification precisely, distinguishes it from adjacent commitments (instinct verification at integration per B2.06, behavioral correctness testing, ad-hoc per-component testing), enumerates the inherited Paper 1 commitments, identifies operational implications for cell birth, modification, mating, and structural reorganization, and provides the operational test that closes the four-note decomposition of B1.03.

1. Why cell-level inheritance verification needs to be formalized as standalone

Paper 2 commits that all of Paper 1's architectural commitments hold at every level of its three-level structure (cell, aspect, Self): substrate-cell boundary, conflict preservation, AI as substrate mediator, tool-agnosticism, linear-cost scaling, and human-governed authority all hold at each level, with level-distinct scope per the recursive-levels principle the source paper defends. A separate derivation note (B1.20) formalizes that recursive inheritance as the architectural claim Paper 2 makes.

The recursive commitment is correct as far as it goes, and this note does not contradict it. But it leaves a class of operational questions architecturally underspecified. How does a deployment *establish* that Paper 1's

commitments actually hold for any specific cell, rather than only that the architecture promises they should? What gates cell birth, modification, and structural integration on commitment-conformance? Without explicit answers, recursive inheritance is claimed but not demonstrable.

The answer Paper 2 deployments use is Series A's standard operational test suite. Paper 1's derivation note series specifies sixteen operational tests (A5.01–A5.16), each verifying one Paper 1 commitment at cell scope. These tests were specified for the cell — Paper 1's atomic unit — and their cell-scope semantics carry forward into Paper 2 unchanged. Paper 2 extends them recursively to aspect and Self levels (separate decompositions cover those extensions under B1.04 and B1.05), but at cell level the suite is the same.

This note formalizes the application of A5.01–A5.16 at cell level in Paper 2 deployments as the operational mechanism establishing the recursive Paper 1 commitments at cell scope. It is the fourteenth Phase B2 note and the closing note of the four-note B1.03 decomposition: B2.11 specifies the cell's external interfaces, B2.12 the cell's internal architecture, B2.13 cell-to-cell relationships within aspects, and this note (B2.14) cell-level inheritance verification.

2. The cell-level verification, defined precisely

Cell-level inheritance verification is the application of Series A's sixteen operational tests (A5.01–A5.16) at cell scope. Each test verifies one Paper 1 commitment for a specific cell as architectural artifact. The tests as applied at cell level:

A5.01 — inspect right test. Verifies humans with appropriate access can read the cell's substrate content and orchestration rules in inspectable form per A2.01.

A5.02 — modify right test. Verifies humans with appropriate access can modify the cell's substrate content and orchestration rules per A2.02.

A5.03 — override right test. Verifies humans can override specific cell decisions, defaults, or LLM-produced outputs per A2.03 without architectural justification gating.

A5.04 — rule authoring test. Verifies the cell's behavior is governed by human-authored orchestration rules per A2.04, rather than by rules emerging from runtime inference or vendor-fixed policy.

A5.05 — mediator role test. Verifies that LLM consultation by the cell respects the five mediator-role properties (Properties A–E from A2.18–A2.23). Under the instinct/reasoning separation per B2.01, the cell's LLM use is the instinct layer; this test verifies it operates in the bounded mediator role rather than as primary controller.

A5.06 — cell-behavior-determinism test. Verifies the cell's behavior given inputs is deterministic per A1.10's determinism contract, with bounded non-determinism allowed only at the points A2.62 specifies.

A5.07 — read-determinism test. Verifies reads against the cell's substrate return consistent results — the same query against the same substrate state returns the same content.

A5.08 — provenance-completeness test. Verifies cell operations carry complete provenance metadata per A2.40 — the six-field accountability vocabulary.

A5.09 — four-accountability-questions test. Verifies that for any cell operation, who initiated it, what was changed, when it occurred, and why are all answerable from substrate content alone.

A5.10 — source-of-truth-five-categories test. Verifies the cell's substrate content fits the five categories (entities, relationships, decisions, rationale, conflicts) specified in A2.42–A2.46.

A5.11 — tool-agnosticism-migration test. Verifies the cell operates correctly when its substrate technology is migrated, per A1.05.

A5.12 — linear-cost-scaling test. Verifies the cell's governance cost scales linearly with rule variety and intervention frequency, not with substrate size, per A1.06.

A5.13 — conflict-coexistence test. Verifies the cell preserves rather than silently merges conflicts per A1.03's two-level conflict handling.

A5.14 — composition-requirements-five test. Verifies the cell satisfies the five composition requirements (A1.13) such that it composes with other cells through substrate as the only channel.

A5.15 — pattern-mapping test. Verifies the cell's relationships to aspects fit one of the three patterns (A/B/C) per A2.92–A2.94.

A5.16 — reproducibility test. Verifies cell operations replay deterministically — given recorded inputs and substrate state, the operation produces the same output, per A1.07 and A1.10 jointly.

The verification is itself governed under Paper 1's authority architecture per A1.01 — humans configure which tests run with which parameters, review verification results, and take corrective action when verification fails. Verification events are recorded as substrate content per A2.40, so the verification record itself carries complete provenance. The same authority architecture that governs the cell governs the cell's verification.

3. What makes cell-level inheritance verification architecturally distinctive

Conventional AI architectures verify component-specific properties when they verify at all. Each component is tested for whatever its designers thought important, with no architectural commitment that the same suite applies across components. The result is verification that is ad-hoc per component and varies across deployments. A component may pass its verification while the system fails to satisfy its architectural commitments, because the component-specific tests do not collectively cover the architectural commitments at all.

Cell-level inheritance verification differs in two architecturally significant respects.

The same standard suite applies recursively. The sixteen tests A5.01–A5.16 are specified once and applied at each level of the cell-aspect-Self hierarchy with level-distinct scope. Cell-level verification uses the suite at cell scope; aspect-level verification (under B1.04) uses it at aspect scope; Self-level verification (under B1.05) uses it at Self scope. Deployments thus have a single verification methodology that applies recursively rather than per-component methodologies that vary. The recursive applicability is what makes Paper 1 inheritance verifiable across the architecture rather than only at the cell level it was originally specified for.

The suite is what makes inheritance demonstrable rather than only claimed. B1.20 commits the architecture to recursive inheritance. B2.14 makes that inheritance operationally demonstrable through the suite's application. Without verification, recursive inheritance is a property the architecture promises but a deployment cannot show. With verification, deployments demonstrate at each cell that Paper 1's commitments hold operationally for that cell, recorded as substrate content with full provenance per A2.40, reviewable by any human with the inspect right per A2.01.

The two together make the recursive inheritance commitment a working architectural property rather than a documented intention.

4. The cognitive analog as conceptual scaffold

The cognitive analog for cell-level inheritance verification is modular component testing in software engineering, where each module is tested for its specific properties under a defined test interface, and the system's correctness is established by the conjunction of module-level verifications under the system's composition rules. The biology analog is loose: tissue-level diagnostics that verify cell function within an organism by applying standardized assays at the cell level (histology, cytometry, single-cell assays) rather than by deriving cell health from organism-level health.

Both analogs function as conceptual scaffold readers absorb quickly because the parallels are intuitive. The architectural substance is the recursive application of a standard test suite at cell scope, rooted in Paper 1's commitments rather than in the analogs. The analogs are how readers orient; the verification's content is what Paper 1 commits to plus what Paper 2 commits to recursively per B1.20.

5. Inherited Paper 1 commitments

Cell-level inheritance verification inherits, rather than introduces, the following Paper 1 commitments:

A1.01 — human-governed. Verification is configured, reviewed, and acted on by humans under the same authority architecture that governs the cell.

A1.07 — path retraceability. Verification events are recorded as substrate content with complete provenance, retraceable through the same accountability vocabulary the cell uses for its own operations.

A1.10 — determinism contract. Each verification test is itself deterministic given inputs — the same test applied to the same cell state with the same parameters returns the same outcome.

A1.13 — composition requirements. Verification covers the five composition requirements directly (via A5.14), so verified cells are demonstrably composable through substrate.

A2.04 — orchestration rule authoring. Verification rules — what is tested, with what parameters, on what schedule, what counts as pass and fail — are themselves human-authored under A2.04's authoring discipline.

A2.40 — provenance metadata. Verification events carry the six-field accountability vocabulary, making verification itself part of the substrate's source-of-truth content.

A5.01–A5.16 — operational test specifications. The tests themselves are inherited from Series A; their cell-scope semantics are unchanged in Paper 2.

B1.20 — recursive Paper 1 commitments. Cell-level verification is the operational mechanism establishing what B1.20 commits to architecturally at cell level.

The list is exhaustive of cell-level inheritance verification's substrate. Anything not on the list is operational concern beyond the verification's architectural scope.

6. Operational implications

Verification is configured per cell type. Different cell types — decision cells, intake cells, summarization cells, lifecycle-management cells, verification cells themselves — may have different parameter values for the standard tests: what counts as "consistent results" for the read-determinism test, what counts as "linear scaling" for the linear-cost test, what counts as "complete provenance" for the provenance-completeness test. The tests are standard; the parameters are deployment-configured under A2.04.

Verification gates cell birth. Per B1.09, newly instantiated cells must pass cell-level verification before becoming operational. Cells that fail verification at birth do not enter operational service until the failures are addressed.

Verification gates cell modification. Per B1.14, cells modified through directed selection re-verify before the modified version replaces the prior version. Modification that introduces a verification failure triggers governance action, not silent acceptance.

Verification supports lifecycle decisions. Per B1.11, cells that fail verification persistently may be subject to functional obsolescence death — retirement of cells that no longer satisfy the architectural commitments the deployment requires.

Verification integrates with mating and vertical evolution. Per B1.10, cells produced through mating (union, selective merge, or lineage-preserved union) must pass cell-level verification before operational integration. Per B1.16, cells whose composition relationships are restructured through vertical evolution re-verify under the new arrangement.

Cross-partner verification follows authority distribution. Per A2.47, when authority over a cell is distributed across partners, verification responsibility follows the same distribution — partners verify cells within their authority scope, and verification results carry the same authority distribution as the cell's content.

Verification failures trigger governance actions. When verification fails, humans use the override right (A2.03) to address the immediate failure or the rule authoring layer (A2.04) to address the underlying configuration. Verification is itself part of the deployment's governance architecture rather than parallel to it.

7. What cell-level inheritance verification is NOT

Cell-level inheritance verification is a precise architectural commitment. Stating what it is not keeps the standalone framing from drifting into something stronger than Paper 2 supports.

Not a replacement for aspect-level or Self-level verification. Aspect-level verification (under B1.04 decomposition) and Self-level verification (under B1.05 decomposition) are distinct levels with distinct scope. The recursive principle says the same suite applies at each level; it does not say that verification at one level discharges verification at another.

Not a replacement for domain-specific cell testing. A cell that verifies under A5.01–A5.16 has demonstrated that Paper 1's architectural commitments hold for it. Whether the cell does its domain job well — whether the loan-decision cell makes good loan decisions, whether the summarization cell summarizes accurately — is operational concern beyond the architectural verification.

Not a prescription of test parameters. The suite specifies what is tested; deployments specify the parameters under which each test is applied. Different cell types and different risk profiles support different parameter choices, all within the suite's scope.

Not a replacement for B2.06 instinct verification gates. B2.06 specifies verification gates for instinct — pre-integration testing of LLM versions and substrate-platform versions before they take effect at the cell. B2.14 specifies verification at cell scope for the cell as architectural artifact under whatever LLM and infrastructure are currently in use. The two are distinct in scope and timing.

Not a guarantee of behavioral correctness. A cell that passes cell-level inheritance verification has demonstrated Paper 1's architectural commitments hold for it. The cell may still produce bad domain outputs; behavioral correctness is operational concern, not architectural commitment.

Not a single test. Cell-level inheritance verification is the application of sixteen tests jointly. A cell that passes some but not all does not satisfy the verification; the suite is jointly necessary, with each test covering a different Paper 1 commitment.

8. Operational test

A Paper 2 deployment establishes that Paper 1 commitments hold at cell level for a specific cell if and only if all of the following are true at all times during the cell's operational existence:

1. The cell passes A5.01–A5.16 at cell scope under deployment-configured parameters, with passing outcomes recorded as substrate content per A2.40.
2. The verification rules specifying what is tested, with what parameters, on what schedule, and what counts as pass or fail are authored under A2.04 and themselves subject to A1.01 governance.
3. Verification gates cell birth (B1.09), cell modification (B1.14), cell mating integration (B1.10), and cell restructuring under vertical evolution (B1.16) such that no cell becomes or remains operational without current passing verification.
4. Verification failures trigger governance action under A2.03 (override) or A2.04 (rule authoring) rather than silent acceptance.
5. Verification responsibility tracks authority distribution per A2.47 across partners holding distributed authority over the cell.

A deployment that fails any of (1)–(5) for a given cell has not established the recursive Paper 1 commitments operationally for that cell, regardless of the deployment's other governance properties.

9. Closing the B1.03 decomposition

The four-note B1.03 decomposition is now complete. B2.11 specifies the cell's external interfaces — what crosses the cell boundary and under what discipline. B2.12 specifies the cell's internal architecture — DNA layer, action layer, harness substrate, and the boundaries among them. B2.13 specifies cell-to-cell relationships within aspects — how cells coordinate through substrate without direct channels. This note (B2.14) specifies cell-level inheritance verification — the operational mechanism by which Paper 1's commitments hold at cell scope, recursively per B1.20.

The four notes together formalize the cell as Paper 2's atomic unit along four operational axes: as bounded modular component (B2.11), as internally-architected artifact (B2.12), as element coordinating with peers through substrate (B2.13), and as architectural artifact verifiable against Paper 1's commitments (B2.14). Subsequent Phase B2 notes (B2.15–B2.19) begin the four-note decomposition of B1.04 (aspect as coordination arrangement of cells), repeating the same operational-decomposition pattern at the next level of Paper 2's three-level structure.

Naming cell-level inheritance verification as a standalone operational variant is what makes the recursive Paper 1 commitments per B1.20 demonstrable in deployment rather than only declared in architecture. Subsequent work that implements, extends, or argues against the verification should treat A5.01–A5.16 at cell scope as the canonical specification of what cell-level inheritance verification requires. Subsequent work that uses a different test suite, a different scope, or a different governance architecture for cell-level verification is implementing a different commitment, and the difference should be named.

— — —

Source papers

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Cell-Level Inheritance Verification: Applying the Standard Operational Test Suite at Cell Scope in CKS-Governed AI Selves*. May 8, 2026. ORCID: 0009-0004-8065-3235.